

Algoritmi avansați

C12 - Intersecții

Mihai-Sorin Stupariu

Sem. al II-lea, 2024 - 2025

Intersecția segmentelor (generalități)

Analiza complexității algebrice

Algoritm - comentarii inițiale

Metoda dreptei de baleiere

Alte rezultate

Subdiviziuni planare. DCEL

Motivație (Probleme geometrice în context 2D)

- **Problema 1.** Dată o listă (mulțime ordonată) de puncte $\mathcal{P} = (P_1, P_2, \dots, P_m)$, să se stabilească dacă ea reprezintă un poligon (linie poligonală fără autointersecții).

Motivație (Probleme geometrice în context 2D)

- ▶ **Problema 1.** Dată o listă (mulțime ordonată) de puncte $\mathcal{P} = (P_1, P_2, \dots, P_m)$, să se stabilească dacă ea reprezintă un poligon (linie poligonală fără autointersecții).
- ▶ **Problema 2.** Date două poligoane $\mathcal{P}$ și $\mathcal{Q}$, să se stabilească dacă se intersectează (interioarele lor se intersectează).

Motivație (Probleme geometrice în context 2D)

- ▶ **Problema 1.** Dată o listă (mulțime ordonată) de puncte $\mathcal{P} = (P_1, P_2, \dots, P_m)$, să se stabilească dacă ea reprezintă un poligon (linie poligonală fără autointersecții).
- ▶ **Problema 2.** Date două poligoane $\mathcal{P}$ și $\mathcal{Q}$, să se stabilească dacă se intersectează (interioarele lor se intersectează).
- ▶ **Problema 3.** Dată o mulțime $\mathcal{S} = \{s_1, \dots, s_n\}$ de n segmente închise din plan, să se determine toate perechile care se intersectează.

Motivație (Probleme geometrice în context 2D)

- ▶ **Problema 1.** Dată o listă (mulțime ordonată) de puncte $\mathcal{P} = (P_1, P_2, \dots, P_m)$, să se stabilească dacă ea reprezintă un poligon (linie poligonală fără autointersecții).
- ▶ **Problema 2.** Date două poligoane $\mathcal{P}$ și $\mathcal{Q}$, să se stabilească dacă se intersectează (interioarele lor se intersectează).
- ▶ **Problema 3.** Dată o mulțime $\mathcal{S} = \{s_1, \dots, s_n\}$ de n segmente închise din plan, să se determine toate perechile care se intersectează.
- ▶ **Problema 3'.** Dată o mulțime $\mathcal{S} = \{s_1, \dots, s_n\}$ de n segmente închise din plan, să se determine toate punctele de intersecție dintre ele.

Determinarea complexității algebrice (I)

- Determinarea complexității algebrice revine la a stabili natura calculelor care trebuie efectuate / a expresiilor care trebuie evaluate pentru a rezolva o problemă (polinoame de un anumit grad, rapoarte de polinoame, etc.). Nu interesează de câte ori se repetă un anumit tip de calcule, ci cât de complexe sunt acestea. Pentru problema analizată considerăm două segmente $[AB]$ și $[CD]$, cu coordonate $A = (x_A, y_A)$, $B = (x_B, y_B)$, $C = (x_C, y_C)$, $D = (x_D, y_D)$.

Determinarea complexității algebrice (I)

- ▶ Determinarea complexității algebrice revine la a stabili natura calculelor care trebuie efectuate / a expresiilor care trebuie evaluate pentru a rezolva o problemă (polinoame de un anumit grad, rapoarte de polinoame, etc.). Nu interesează de câte ori se repetă un anumit tip de calcule, ci cât de complexe sunt acestea. Pentru problema analizată considerăm două segmente $[AB]$ și $[CD]$, cu coordonate $A = (x_A, y_A)$, $B = (x_B, y_B)$, $C = (x_C, y_C)$, $D = (x_D, y_D)$.
- ▶ Dacă segmentele $[AB]$ și $[CD]$ nu sunt incluse în aceeași dreaptă, atunci ele se intersectează $\Leftrightarrow A$ și B sunt de o parte și de alta a lui CD și $\Leftrightarrow C$ și D sunt de o parte și de alta a lui AB . Aceasta revine la a aplica testul de orientare $\rightarrow$ **polinoame de gradul II**.
- ▶ De stabilit cum se procedează dacă segmentele sunt incluse în aceeași dreaptă.

Determinarea complexității algebrice (IIa)

- Pentru a **determina explicit** punctul de intersecție dintre două segmente $[AB]$ și $[CD]$: un punct M este atât pe segmentul $[AB]$, cât și pe segmentul $[CD]$ $\Leftrightarrow$ există $\lambda, \mu \in [0, 1]$ astfel ca

$$M = (1 - \lambda)A + \lambda B = (1 - \mu)C + \mu D \quad (1)$$

(am scris punctul M ca fiind atât o combinație convexă a lui A și B , cât și o combinație convexă a lui C și D).

Determinarea complexității algebrice (IIa)

- Pentru a **determina explicit** punctul de intersecție dintre două segmente $[AB]$ și $[CD]$: un punct M este atât pe segmentul $[AB]$, cât și pe segmentul $[CD]$ $\Leftrightarrow$ există $\lambda, \mu \in [0, 1]$ astfel ca

$$M = (1 - \lambda)A + \lambda B = (1 - \mu)C + \mu D \quad (1)$$

(am scris punctul M ca fiind atât o combinație convexă a lui A și B , cât și o combinație convexă a lui C și D).

- Ecuația (1) se scrie în coordonate

$$\begin{cases} x_M = (1 - \lambda)x_A + \lambda x_B = (1 - \mu)x_C + \mu x_D \\ y_M = (1 - \lambda)y_A + \lambda y_B = (1 - \mu)y_C + \mu y_D \end{cases} \quad (2)$$

Am obținut sistemul (2) de două ecuații cu două necunoscute (λ, μ) , care poate fi rescris sub forma

$$\begin{cases} (x_B - x_A)\lambda + (x_D - x_C)\mu = x_C - x_A \\ (y_B - y_A)\lambda + (y_D - y_C)\mu = y_C - y_A \end{cases} \quad (3)$$

Determinarea complexității algebrice (IIa)

- Pentru sistemul (2), determinantul

$$\Delta = \begin{vmatrix} x_B - x_A & x_D - x_C \\ y_B - y_A & y_D - y_C \end{vmatrix} = (x_B - x_A)(y_D - y_C) - (y_B - y_A)(x_D - x_C)$$

este un polinom de gradul II. Dreptele AB și CD se intersectează într-un singur punct $\Leftrightarrow \Delta \neq 0$.

Determinarea complexității algebrice (IIa)

- Pentru sistemul (2), determinantul

$$\Delta = \begin{vmatrix} x_B - x_A & x_D - x_C \\ y_B - y_A & y_D - y_C \end{vmatrix} = (x_B - x_A)(y_D - y_C) - (y_B - y_A)(x_D - x_C)$$

este un polinom de gradul II. Dreptele AB și CD se intersectează într-un singur punct $\Leftrightarrow \Delta \neq 0$.

- Dacă $\Delta \neq 0$, atunci

$$\lambda = \frac{1}{\Delta} \begin{vmatrix} x_C - x_A & x_D - x_C \\ y_C - y_A & y_D - y_C \end{vmatrix}, \quad \mu = \frac{1}{\Delta} \begin{vmatrix} x_B - x_A & x_C - x_A \\ y_B - y_A & y_C - y_A \end{vmatrix},$$

deci calculul soluțiilor λ și μ ale sistemului (2) revine la evaluarea unor rapoarte de forma $\frac{\text{polinom de gradul II}}{\text{polinom de gradul II}}$. Mai trebuie verificat faptul că $\lambda, \mu \in [0, 1]$.

Determinarea complexității algebrice (IIa)

- Pentru sistemul (2), determinantul

$$\Delta = \begin{vmatrix} x_B - x_A & x_D - x_C \\ y_B - y_A & y_D - y_C \end{vmatrix} = (x_B - x_A)(y_D - y_C) - (y_B - y_A)(x_D - x_C)$$

este un polinom de gradul II. Dreptele AB și CD se intersectează într-un singur punct $\Leftrightarrow \Delta \neq 0$.

- Dacă $\Delta \neq 0$, atunci

$$\lambda = \frac{1}{\Delta} \begin{vmatrix} x_C - x_A & x_D - x_C \\ y_C - y_A & y_D - y_C \end{vmatrix}, \quad \mu = \frac{1}{\Delta} \begin{vmatrix} x_B - x_A & x_C - x_A \\ y_B - y_A & y_C - y_A \end{vmatrix},$$

deci calculul soluțiilor λ și μ ale sistemului (2) revine la evaluarea unor rapoarte de forma $\frac{\text{polinom de gradul II}}{\text{polinom de gradul II}}$. Mai trebuie verificat faptul că $\lambda, \mu \in [0, 1]$.

- În final, prin înlocuirea lui λ și μ în sistemul (2), se găsesc coordonatele punctului de intersecție $\rightarrow \frac{\text{polinom de gradul III}}{\text{polinom de gradul II}}$.

Algoritmul trivial

- ▶ **Idee de lucru:** Sunt considerate toate perechile de segmente și se determină cele care se intersectează / se calculează punctele de intersecție.

Algoritmul trivial

- ▶ **Idee de lucru:** Sunt considerate toate perechile de segmente și se determină cele care se intersectează / se calculează punctele de intersecție.
- ▶ **Complexitate:**

Algoritmul trivial

- ▶ **Idee de lucru:** Sunt considerate toate perechile de segmente și se determină cele care se intersectează / se calculează punctele de intersecție.
- ▶ **Complexitate:**
 - ▶ timp: $O(n^2)$

Algoritmul trivial

- ▶ **Idee de lucru:** Sunt considerate toate perechile de segmente și se determină cele care se intersectează / se calculează punctele de intersecție.
- ▶ **Complexitate:**
 - ▶ timp: $O(n^2)$
 - ▶ memorie: $O(n)$

Algoritmul trivial

- ▶ **Idee de lucru:** Sunt considerate toate perechile de segmente și se determină cele care se intersectează / se calculează punctele de intersecție.
- ▶ **Complexitate:**
 - ▶ timp: $O(n^2)$
 - ▶ memorie: $O(n)$
 - ▶ algebric: polinoame de gradul II (Problema 3), rapoarte de polinoame (Problema 3')

Algoritmul trivial

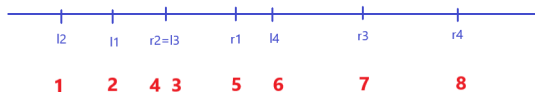
- ▶ **Idee de lucru:** Sunt considerate toate perechile de segmente și se determină cele care se intersectează / se calculează punctele de intersecție.
- ▶ **Complexitate:**
 - ▶ timp: $O(n^2)$
 - ▶ memorie: $O(n)$
 - ▶ algebric: polinoame de gradul II (Problema 3), rapoarte de polinoame (Problema 3')
- ▶ **Comentariu:** În anumite cazuri: optim (dacă toate segmentele se intersectează).

Algoritmul trivial

- ▶ **Idee de lucru:** Sunt considerate toate perechile de segmente și se determină cele care se intersectează / se calculează punctele de intersecție.
- ▶ **Complexitate:**
 - ▶ timp: $O(n^2)$
 - ▶ memorie: $O(n)$
 - ▶ algebric: polinoame de gradul II (Problema 3), rapoarte de polinoame (Problema 3')
- ▶ **Comentariu:** În anumite cazuri: optim (dacă toate segmentele se intersectează).
- ▶ Algoritmi mai eficienți (*output / intersection sensitive*)?

Rezolvarea problemei în context 1D

- Se dau segmentele $[l_i, r_i]$ ($i = 1, \dots, n$). Mai întâi este realizată ordonarea lexicografică a extremităților segmentelor / intervalelor într-o listă $\mathcal{P}$.



- Lista $\mathcal{P}$ este parcursă (crescător); lista $\mathcal{L}$ a segmentelor care conțin punctul curent din $\mathcal{P}$ este actualizată:

Rezolvarea problemei în context 1D

- Se dau segmentele $[l_i, r_i]$ ($i = 1, \dots, n$). Mai întâi este realizată ordonarea lexicografică a extremităților segmentelor / intervalelor într-o listă $\mathcal{P}$.



- Lista $\mathcal{P}$ este parcursă (crescător); lista $\mathcal{L}$ a segmentelor care conțin punctul curent din $\mathcal{P}$ este actualizată:
 - dacă punctul curent este marginea din stânga a unui segment s , atunci s este adăugat la listă $\mathcal{L}$
 - dacă punctul curent este marginea din dreapta a unui segment s , atunci s este șters din $\mathcal{L}$ și se raportează intersecții între s și toate segmentele din $\mathcal{L}$

Rezolvarea problemei în context 1D

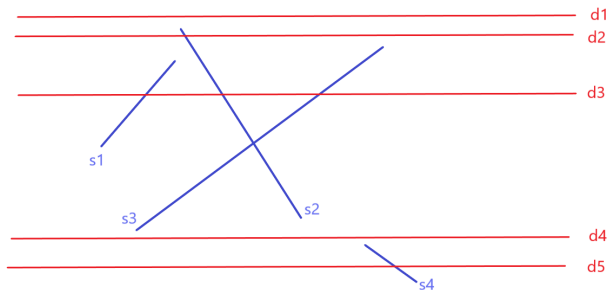
- Se dau segmentele $[l_i, r_i]$ ($i = 1, \dots, n$). Mai întâi este realizată ordonarea lexicografică a extremităților segmentelor / intervalelor într-o listă $\mathcal{P}$.



- Lista $\mathcal{P}$ este parcursă (crescător); lista $\mathcal{L}$ a segmentelor care conțin punctul curent din $\mathcal{P}$ este actualizată:
 - dacă punctul curent este marginea din stânga a unui segment s , atunci s este adăugat la listă $\mathcal{L}$
 - dacă punctul curent este marginea din dreapta a unui segment s , atunci s este șters din $\mathcal{L}$ și se raportează intersecții între s și toate segmentele din $\mathcal{L}$
- Teoremă.** Algoritmul are complexitate $O(n \log n + k)$ și necesită $O(n)$ memorie (k este numărul de perechi ce se intersectează).

Un prim algoritm

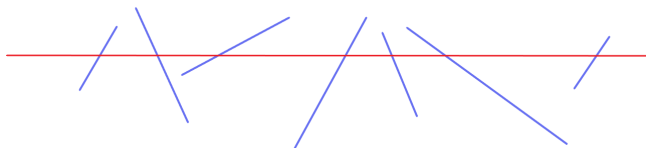
Dreapta de baleiere d : **orizontală**, astfel că toate intersecțiile situate deasupra dreptei de baleiere au fost detectate. **Statutul**: mulțimea segmentelor care intersectează d . Pentru aceste segmente se testează efectiv dacă se intersectează sau nu. **Evenimente** : capetele segmentelor $\rightarrow$ actualizarea statutului.



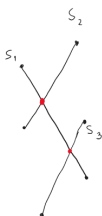
Statutul dreptelor din figură. $d_1 : \emptyset$; $d_2 : s_2$; $d_3 : s_1, s_2, s_3$; $d_4 : \emptyset$; $d_5 : s_4$.

Un prim algoritm

Obs. Sunt testate pentru intersecție segmente care intersectează, la un moment dat, dreapta de baleiere $d \rightarrow$ (sunt testate segmentele care sunt "aproape" de-a lungul axei Oy) $\rightarrow$ încă inefficient.



Statut - posibile abordări



Varianta 1

Statutul = mulțime

Eveniment	Statut
u_{s_2}	$\emptyset$
u_{s_1}	$\{s_2\}$
u_{s_3}	$\{s_1, s_2, s_3\}$
d_{s_2}	$\{s_1, s_3\}$
d_{s_1}	$\{s_3\}$
d_{s_3}	$\emptyset$

Varianta 2

Statutul = listă
(mulțime ordonată)

Eveniment	Statut
u_{s_2}	$\emptyset$
u_{s_1}	(s_2)
u_{s_3}	(s_1, s_2)
d_{s_2}	(s_2, s_1)
d_{s_1}	(s_2, s_1, s_3)
d_{s_3}	$\emptyset$

insert

Handwritten annotations in red:

- Arrows pointing from the u_{s_1} and u_{s_3} rows to the d_{s_2} row, indicating insertion.
- Text: $s_1, n_{s_2}?$ (next to u_{s_1})
- Text: $s_1, n_{s_3}?$ (next to u_{s_3})
- Text: $s_1, n_{s_2}?$ (next to d_{s_2})
- Text: $s_1, n_{s_3}?$ (next to d_{s_1})

Pentru segmentele s_1, s_2, s_3 din figură sunt prezentate două variante pentru statut. Marginea superioară a segmentului s este notată cu u_s , iar cea inferioară cu d_s .

În tabelul din stânga statutul este o mulțime (nu se ține cont de ordinea segmentelor), iar în tabelul din dreapta statutul este o listă (o mulțime ordonată, ordinea elementelor fiind relevantă). În varianta 2 (statutul=listă), punctele de intersecție devin și ele evenimente și trebuie inserate în coada de evenimente.

Un algoritm eficient [Bentley și Ottmann, 1979; Mehlhorn și Näher, 1994]

- **Idee de lucru:** ordonare (segmente, evenimente).

Un algoritm eficient [Bentley și Ottmann, 1979; Mehlhorn și Näher, 1994]

- ▶ **Idee de lucru:** ordonare (segmente, evenimente).
 - ▶ Segmentele: ordonate folosind extremitățile superioare (lexicografic, x apoi y).

Un algoritm eficient [Bentley și Ottmann, 1979; Mehlhorn și Näher, 1994]

- ▶ **Idee de lucru:** ordonare (segmente, evenimente).
 - ▶ Segmentele: ordonate folosind extremitățile superioare (lexicografic, x apoi y).
 - ▶ Evenimente (puncte): (lexicografic, y apoi x).

Un algoritm eficient [Bentley și Ottmann, 1979; Mehlhorn și Näher, 1994]

- ▶ **Idee de lucru:** ordonare (segmente, evenimente).
 - ▶ Segmentele: ordonate folosind extremitățile superioare (lexicografic, x apoi y).
 - ▶ Evenimente (puncte): (lexicografic, y apoi x).
 - ▶ Statutul: **listă** (mulțime ordonată)

Un algoritm eficient [Bentley și Ottmann, 1979; Mehlhorn și Näher, 1994]

- ▶ **Idee de lucru:** ordonare (segmente, evenimente).
 - ▶ Segmentele: ordonate folosind extremitățile superioare (lexicografic, x apoi y).
 - ▶ Evenimente (puncte): (lexicografic, y apoi x).
 - ▶ Statutul: **listă** (mulțime ordonată)
- ▶ **Avantaj:** în momentul modificării statutului, sunt testate intersecțiile doar în raport cu vecinii din listă (sunt testate segmentele care sunt "aproape" de-a lungul axei Ox).

Un algoritm eficient [Bentley și Ottmann, 1979; Mehlhorn și Näher, 1994]

- ▶ **Idee de lucru:** ordonare (segmente, evenimente).
 - ▶ Segmente: ordonate folosind extremitățile superioare (lexicografic, x apoi y).
 - ▶ Evenimente (puncte): (lexicografic, y apoi x).
 - ▶ Statutul: **listă** (mulțime ordonată)
- ▶ **Avantaj:** în momentul modificării statutului, sunt testate intersecțiile doar în raport cu vecinii din listă (sunt testate segmentele care sunt "aproape" de-a lungul axei Ox).
- ▶ **Fundamental:** Punctele de intersecție devin, la rândul lor, evenimente, deoarece schimbă ordinea segmenetelor care le determină
 → chiar dacă nu sunt determinate explicit, trebuie inserate în lista de evenimente (compararea coordonatelor poate necesita utilizarea unor polinoame de gradul V)

3 tipuri de evenimente

- ▶ Marginea superioară a unui segment
 - ▶ apare un nou segment în statutul liniei de baleiere, ce trebuie inserat corespunzător
 - ▶ testat, în raport cu vecinii, dacă au puncte de intersecție sub linia de baleiere → vor deveni ulterior evenimente

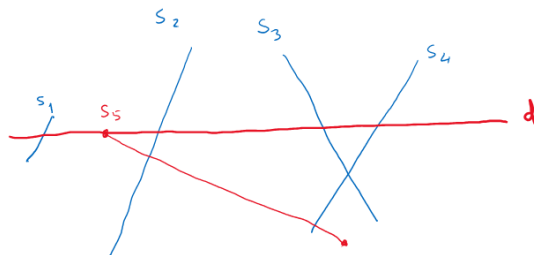
3 tipuri de evenimente

- ▶ Marginea superioară a unui segment
 - ▶ apare un nou segment în statutul liniei de baleiere, ce trebuie inserat corespunzător
 - ▶ testat, în raport cu vecinii, dacă au puncte de intersecție sub linia de baleiere → vor deveni ulterior evenimente
- ▶ Marginea inferioară a unui segment
 - ▶ eliminat un segment din statutul liniei de baleiere
 - ▶ testare pentru segmentele vecine nou apărute

3 tipuri de evenimente

- ▶ Marginea superioară a unui segment
 - ▶ apare un nou segment în statutul liniei de baleiere, ce trebuie inserat corespunzător
 - ▶ testat, în raport cu vecinii, dacă au puncte de intersecție sub linia de baleiere → vor deveni ulterior evenimente
- ▶ Marginea inferioară a unui segment
 - ▶ eliminat un segment din statutul liniei de baleiere
 - ▶ testare pentru segmentele vecine nou apărute
- ▶ punctele de intersecție (inserate în mod corespunzător pe parcurs)
 - ▶ segmentele care le determină trebuie "inversate" în statutul liniei de baleiere
 - ▶ testare pentru segmentele vecine nou apărute

Evenimente de tip margine superioară



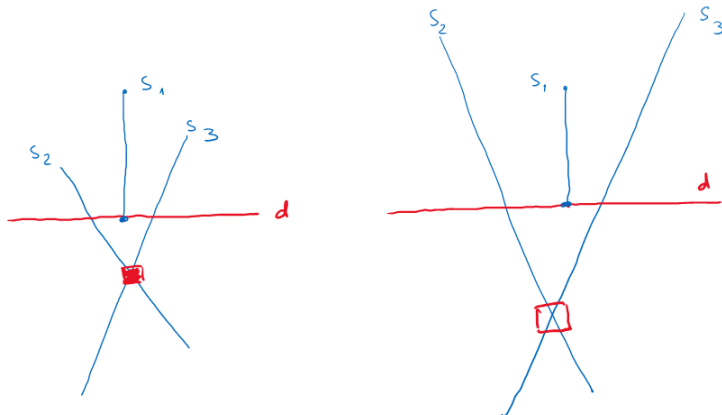
Evenimentul u_{s_5} :

(s_1, s_2, s_3, s_4)

$\downarrow u_{s_5}$
 $(s_1, \underline{s_5}, s_2, s_3, s_4)$

La evenimentul u_{s_5} statutul se modifică de la (s_1, s_2, s_3, s_4) la $(s_1, \underline{s_5}, s_2, s_3, s_4)$. De asemenea, sunt efectuate teste de intersecție **între vecinii nou apăruiți**. Astfel, s_1 și s_5 nu se intersectează, în schimb s_5 și s_2 se intersectează, fiind inserat un nou eveniment $(s_5 \cap s_2)$.

Evenimente de tip margine inferioară



Evenimentul analizat este d_{s_1} . În figura din stânga, evenimentul $s_2 \cap s_3$ nu a fost detectat anterior și este adăugat în lista de evenimente. În figura din dreapta, evenimentul $s_2 \cap s_3$ a fost detectat anterior, întrucât s_2 și s_3 au fost vecine în statul înainte de evenimentul u_{s_1} .

Implementare: structuri de date utilizate

- ▶ Q – coada/lista de evenimente (*event queue*)

Implementare: structuri de date utilizate

- ▶ Q – **coada/lista de evenimente** (*event queue*)
 - ▶ puncte, ordonate lexicografic, după y apoi x ; memorată într-un arbore de căutare binar echilibrat (*balanced binary search tree*)

Implementare: structuri de date utilizate

- ▶ Q – **coada/lista de evenimente** (*event queue*)
 - ▶ puncte, ordonate lexicografic, după y apoi x ; memorată într-un arbore de căutare binar echilibrat (*balanced binary search tree*)
 - ▶ evenimentele nou detectate trebuie inserate în mod corespunzător!

Implementare: structuri de date utilizate

- ▶ Q – **coada/lista de evenimente** (*event queue*)
 - ▶ puncte, ordonate lexicografic, după y apoi x ; memorată într-un arbore de căutare binar echilibrat (*balanced binary search tree*)
 - ▶ evenimentele nou detectate trebuie inserate în mod corespunzător!
 - ▶ de evaluat: complexitatea-timp (pentru o inserare, numărul de repetiții); complexitatea spațiu

Implementare: structuri de date utilizate

- ▶ $\mathcal{Q}$ – **coada/lista de evenimente** (*event queue*)
 - ▶ puncte, ordonate lexicografic, după y apoi x ; memorată într-un arbore de căutare binar echilibrat (*balanced binary search tree*)
 - ▶ evenimentele nou detectate trebuie inserate în mod corespunzător!
 - ▶ de evaluat: complexitatea-timp (pentru o inserare, numărul de repetiții); complexitatea spațiu
- ▶ $\mathcal{T}$ – **statut**: arbore de căutare binar echilibrat

Implementare: structuri de date utilizate

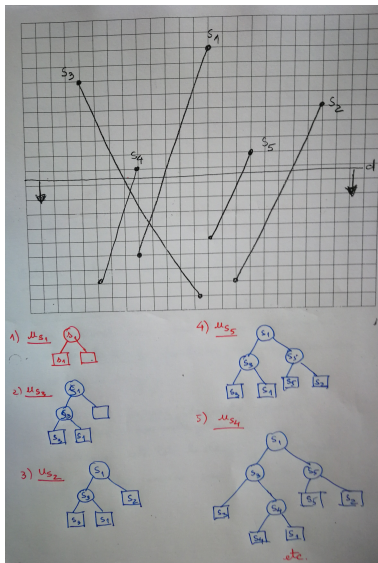
- ▶ $\mathcal{Q}$ – **coada/lista de evenimente** (*event queue*)
 - ▶ puncte, ordonate lexicografic, după y apoi x ; memorată într-un arbore de căutare binar echilibrat (*balanced binary search tree*)
 - ▶ evenimentele nou detectate trebuie inserate în mod corespunzător!
 - ▶ de evaluat: complexitatea-timp (pentru o inserare, numărul de repetiții); complexitatea spațiu
- ▶ $\mathcal{T}$ – **statut**: arbore de căutare binar echilibrat
 - ▶ pe frunze: segmente, este reținută ordinea segmentelor de la stânga la dreapta

Implementare: structuri de date utilizate

- ▶ $\mathcal{Q}$ – **coada/lista de evenimente** (*event queue*)
 - ▶ puncte, ordonate lexicografic, după y apoi x ; memorată într-un arbore de căutare binar echilibrat (*balanced binary search tree*)
 - ▶ evenimentele nou detectate trebuie inserate în mod corespunzător!
 - ▶ de evaluat: complexitatea-timp (pentru o inserare, numărul de repetiții); complexitatea spațiu
- ▶ $\mathcal{T}$ – **statut**: arbore de căutare binar echilibrat
 - ▶ pe frunze: segmente, este reținută ordinea segmentelor de la stânga la dreapta
 - ▶ în nodurile interne: segmente, privite ca elemente de ghidare

Implementare: structuri de date utilizate

- ▶ $\mathcal{Q}$ – **coada/lista de evenimente** (*event queue*)
 - ▶ puncte, ordonate lexicografic, după y apoi x ; memorată într-un arbore de căutare binar echilibrat (*balanced binary search tree*)
 - ▶ evenimentele nou detectate trebuie inserate în mod corespunzător!
 - ▶ de evaluat: complexitatea-timp (pentru o inserare, numărul de repetiții); complexitatea spațiu
- ▶ $\mathcal{T}$ – **statut**: arbore de căutare binar echilibrat
 - ▶ pe frunze: segmente, este reținută ordinea segmentelor de la stânga la dreapta
 - ▶ în nodurile interne: segmente, privite ca elemente de ghidare
 - ▶ de evaluat: complexitatea-timp (pentru o actualizare, numărul de repetiții)



Algoritm INTERSECTII

- **Input.** O mulțime de segmente din planul $\mathbb{R}^2$.

Algoritm INTERSECȚII

- ▶ **Input.** O mulțime de segmente din planul $\mathbb{R}^2$.
- ▶ **Output.** Mulțimea punctelor de intersecție (explicit sau doar formal); pentru fiecare punct precizează segmentele pe care se găsește.

Algoritm INTERSECȚII

- ▶ **Input.** O mulțime de segmente din planul $\mathbb{R}^2$.
 - ▶ **Output.** Mulțimea punctelor de intersecție (explicit sau doar formal); pentru fiecare punct precizează segmentele pe care se găsește.
1. $Q \leftarrow \emptyset$. Inserează extremitățile segmentelor în Q ; împreună cu marginea superioară a unui segment memorează și segmentul

Algoritm INTERSECȚII

- ▶ **Input.** O mulțime de segmente din planul $\mathbb{R}^2$.
 - ▶ **Output.** Mulțimea punctelor de intersecție (explicit sau doar formal); pentru fiecare punct precizează segmentele pe care se găsește.
1. $\mathcal{Q} \leftarrow \emptyset$. Inserează extremitățile segmentelor în $\mathcal{Q}$; împreună cu marginea superioară a unui segment memorează și segmentul
 2. $\mathcal{T} \leftarrow \emptyset$.

Algoritm INTERSECȚII

- ▶ **Input.** O mulțime de segmente din planul $\mathbb{R}^2$.
 - ▶ **Output.** Mulțimea punctelor de intersecție (explicit sau doar formal); pentru fiecare punct precizează segmentele pe care se găsește.
1. $\mathcal{Q} \leftarrow \emptyset$. Inserează extremitățile segmentelor în $\mathcal{Q}$; împreună cu marginea superioară a unui segment memorează și segmentul
 2. $\mathcal{T} \leftarrow \emptyset$.
 3. **while** $\mathcal{Q} \neq \emptyset$

Algoritm INTERSECȚII

- ▶ **Input.** O mulțime de segmente din planul $\mathbb{R}^2$.
 - ▶ **Output.** Mulțimea punctelor de intersecție (explicit sau doar formal); pentru fiecare punct precizează segmentele pe care se găsește.
1. $Q \leftarrow \emptyset$. Inserează extremitățile segmentelor în Q ; împreună cu marginea superioară a unui segment memorează și segmentul
 2. $\mathcal{T} \leftarrow \emptyset$.
 3. **while** $Q \neq \emptyset$
 4. **do** determină evenimentul p care urmează în Q și îl șterge

Algoritm INTERSECȚII

- ▶ **Input.** O mulțime de segmente din planul $\mathbb{R}^2$.
 - ▶ **Output.** Mulțimea punctelor de intersecție (explicit sau doar formal); pentru fiecare punct precizează segmentele pe care se găsește.
1. $Q \leftarrow \emptyset$. Inserează extremitățile segmentelor în Q ; împreună cu marginea superioară a unui segment memorează și segmentul
 2. $\mathcal{T} \leftarrow \emptyset$.
 3. **while** $Q \neq \emptyset$
 4. **do** determină evenimentul p care urmează în Q și îl șterge
 5. ANALIZEAZA (p)

ANALIZEAZA (p)

1. Fie $U(p)$ mulțimea segmentelor a căror extremitate superioară este p (pentru cele orizontale este marginea din stânga) - stocată cu p în Q .

ANALIZEAZA (p)

1. Fie $U(p)$ mulțimea segmentelor a căror extremitate superioară este p (pentru cele orizontale este marginea din stânga) - stocată cu p în Q .
2. Determină toate segmentele care conțin p : sunt adiacente în $\mathcal{T}$ (de ce?) Fie $D(p)$, respectiv $Int(p)$, mulțimea segmentelor care au p drept margine inferioară, respectiv conțin p în interior.

ANALIZEAZA (p)

1. Fie $U(p)$ mulțimea segmentelor a căror extremitate superioară este p (pentru cele orizontale este marginea din stânga) - stocată cu p în Q .
2. Determină toate segmentele care conțin p : sunt adiacente în $\mathcal{T}$ (de ce?) Fie $D(p)$, respectiv $Int(p)$, mulțimea segmentelor care au p drept margine inferioară, respectiv conțin p în interior.
3. **if** $U(p) \cup D(p) \cup Int(p)$ conține mai mult de un segment

ANALIZEAZA (p)

1. Fie $U(p)$ mulțimea segmentelor a căror extremitate superioară este p (pentru cele orizontale este marginea din stânga) - stocată cu p în Q .
2. Determină toate segmentele care conțin p : sunt adiacente în $\mathcal{T}$ (de ce?) Fie $D(p)$, respectiv $Int(p)$, mulțimea segmentelor care au p drept margine inferioară, respectiv conțin p în interior.
3. **if** $U(p) \cup D(p) \cup Int(p)$ conține mai mult de un segment
4. **then** raportează p ca punct de intersecție, împreună cu segmentele din $U(p)$, $D(p)$, $Int(p)$

ANALIZEAZA (p)

5. șterge segmentele din $D(p) \cup Int(p)$ din $\mathcal{T}$

ANALIZEAZA (p)

5. șterge segmentele din $D(p) \cup Int(p)$ din $\mathcal{T}$
6. inserează segmentele din $U(p) \cup Int(p)$ în $\mathcal{T}$ (ordinea segmentelor pe frunzele lui $\mathcal{T}$ coincide cu ordinea în care sunt intersectate de o linie de baleiere situată imediat sub p)

ANALIZEAZA (p)

7. **if** $U(p) \cup Int(p) = \emptyset$

ANALIZEAZA (p)

7. **if** $U(p) \cup Int(p) = \emptyset$
8. **then** fie s_l și s_r vecinii din stânga/dreapta ai lui p din $\mathcal{T}$

ANALIZEAZA (p)

7. **if** $U(p) \cup Int(p) = \emptyset$
8. **then** fie s_l și s_r vecinii din stânga/dreapta ai lui p din $\mathcal{T}$
9. DETERMINAEVENIMENT (s_l, s_r, p)

ANALIZEAZA (p)

7. **if** $U(p) \cup Int(p) = \emptyset$
8. **then** fie s_l și s_r vecinii din stânga/dreapta ai lui p din $\mathcal{T}$
9. DETERMINAEVENIMENT (s_l, s_r, p)
10. **else** fie s' din $U(p) \cup Int(p)$ cel mai în stânga în $\mathcal{T}$

ANALIZEAZA (p)

7. **if** $U(p) \cup Int(p) = \emptyset$
8. **then** fie s_l și s_r vecinii din stânga/dreapta ai lui p din $\mathcal{T}$
9. DETERMINAEVENIMENT (s_l, s_r, p)
10. **else** fie s' din $U(p) \cup Int(p)$ cel mai în stânga în $\mathcal{T}$
11. fie s_l vecinul din stânga al lui p

ANALIZEAZA (p)

7. **if** $U(p) \cup Int(p) = \emptyset$
8. **then** fie s_l și s_r vecinii din stânga/dreapta ai lui p din $\mathcal{T}$
9. DETERMINAEVENIMENT (s_l, s_r, p)
10. **else** fie s' din $U(p) \cup Int(p)$ cel mai în stânga în $\mathcal{T}$
11. fie s_l vecinul din stânga al lui p
12. DETERMINAEVENIMENT (s_l, s', p)

ANALIZEAZA (p)

7. **if** $U(p) \cup Int(p) = \emptyset$
8. **then** fie s_l și s_r vecinii din stânga/dreapta ai lui p din $\mathcal{T}$
9. DETERMINAEVENIMENT (s_l, s_r, p)
10. **else** fie s' din $U(p) \cup Int(p)$ cel mai în stânga în $\mathcal{T}$
11. fie s_l vecinul din stânga al lui p
12. DETERMINAEVENIMENT (s_l, s', p)
13. fie s'' din $U(p) \cup Int(p)$ cel mai în dreapta în $\mathcal{T}$

ANALIZEAZA (p)

7. **if** $U(p) \cup Int(p) = \emptyset$
8. **then** fie s_l și s_r vecinii din stânga/dreapta ai lui p din $\mathcal{T}$
9. DETERMINAEVENIMENT (s_l, s_r, p)
10. **else** fie s' din $U(p) \cup Int(p)$ cel mai în stânga în $\mathcal{T}$
11. fie s_l vecinul din stânga al lui p
12. DETERMINAEVENIMENT (s_l, s', p)
13. fie s'' din $U(p) \cup Int(p)$ cel mai în dreapta în $\mathcal{T}$
14. fie s_r vecinul din dreapta al lui p

ANALIZEAZA (p)

7. **if** $U(p) \cup Int(p) = \emptyset$
8. **then** fie s_l și s_r vecinii din stânga/dreapta ai lui p din $\mathcal{T}$
9. DETERMINAEVENIMENT (s_l, s_r, p)
10. **else** fie s' din $U(p) \cup Int(p)$ cel mai în stânga în $\mathcal{T}$
11. fie s_l vecinul din stânga al lui p
12. DETERMINAEVENIMENT (s_l, s', p)
13. fie s'' din $U(p) \cup Int(p)$ cel mai în dreapta în $\mathcal{T}$
14. fie s_r vecinul din dreapta al lui p
15. DETERMINAEVENIMENT (s'', s_r, p)

DETERMINAEVENIMENT (sgt_l, sgt_r, p)

1. **if** sgt_l și sgt_r se intersectează sub linia de baleiere sau pe linia de baleiere, dar la dreapta lui p și punctul de intersecție nu este deja în Q

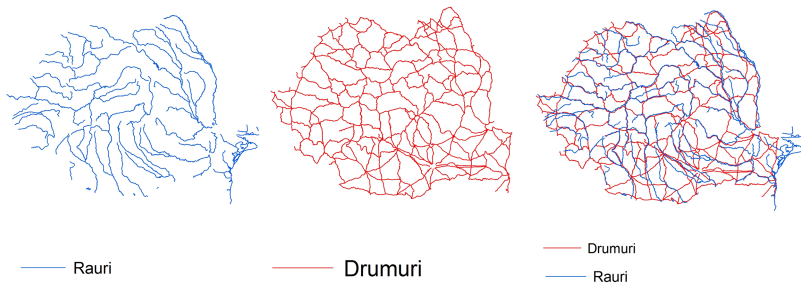
DETERMINAEVENIMENT (sgt_l, sgt_r, p)

1. **if** sgt_l și sgt_r se intersectează sub linia de baleiere sau pe linia de baleiere, dar la dreapta lui p și punctul de intersecție nu este deja în Q
2. **then** inserează punctul de intersecție ca eveniment în Q

Rezultatul principal (intersecții de segmente)

Teoremă. *Fie S o mulțime care conține n segmente din planul $\mathbb{R}^2$. Toate punctele de intersecție ale segmentelor din S , împreună cu segmentele corespunzătoare, pot fi determinate în $O(n \log n + I \log n)$ timp, folosind $O(n)$ spațiu (I este numărul punctelor de intersecție).*

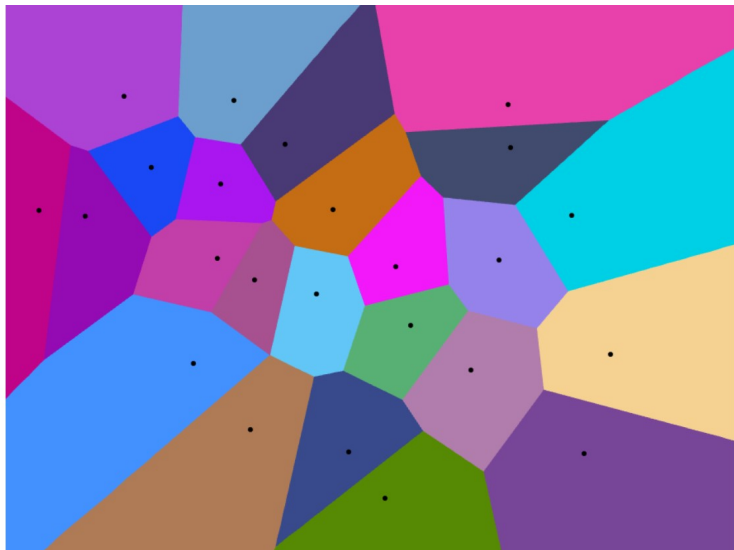
Suprapunerea unor segmente cu conținut tematic diferit



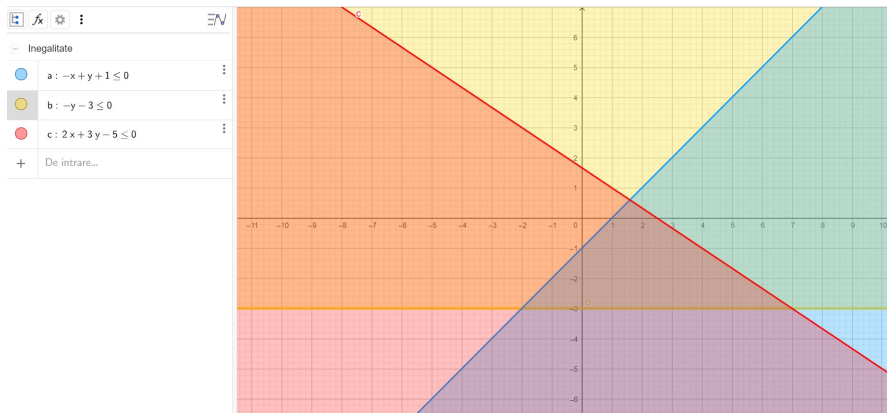
Red-blue intersections [Mairson și Stolfi, 1988; Mantler și Snoeyink, 2000]

Teoremă. Pentru două mulțimi R și B de segmente din $\mathbb{R}^2$ având interioare disjuncte, perechile de segmente ce se intersectează pot fi determinate în $O(n \log n + k)$ timp și spațiu liniar, folosind predicate (polinoame) de grad cel mult 11 ($n = |R| + |B|$) și k este numărul perechilor ce se intersectează).

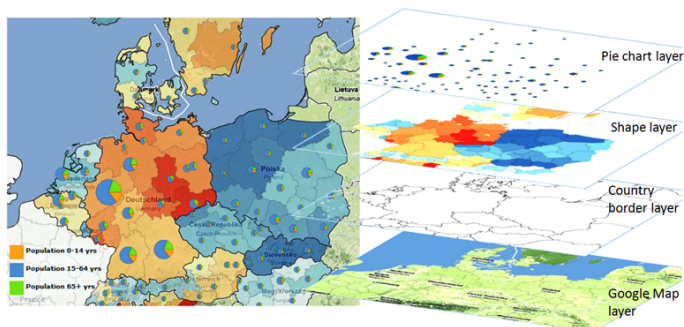
Motivație - reprezentarea diagramelor Voronoi



Motivație - reprezentarea intersecțiilor de semiplane



Motivație - reprezentarea datelor geo-spațiale



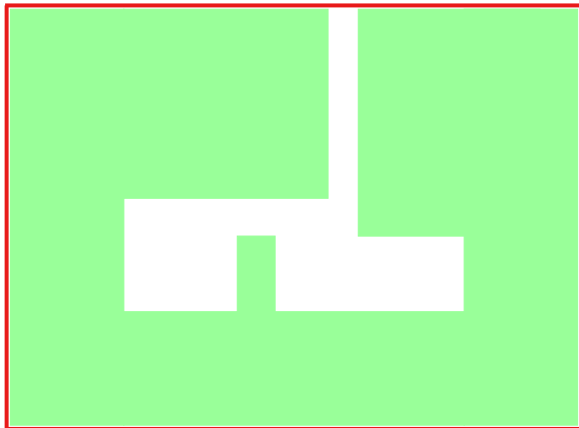
Sursa: <https://s-media-cache-ak0.pinimg.com/originals/37/90/86/37908600ab7db99c424c3bc6e1ddb740.jpg>

Ce structură de date este adecvată pentru a memora astfel de informații?

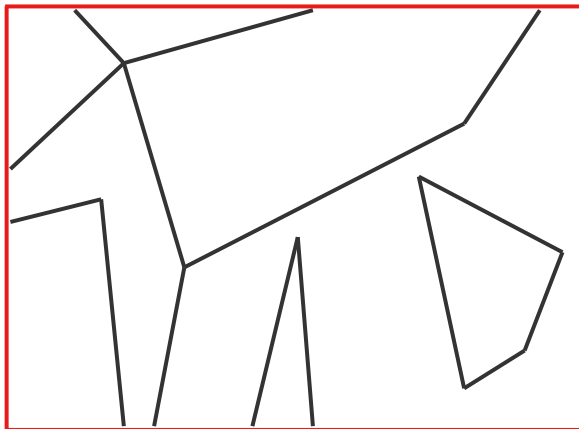
Problematizare



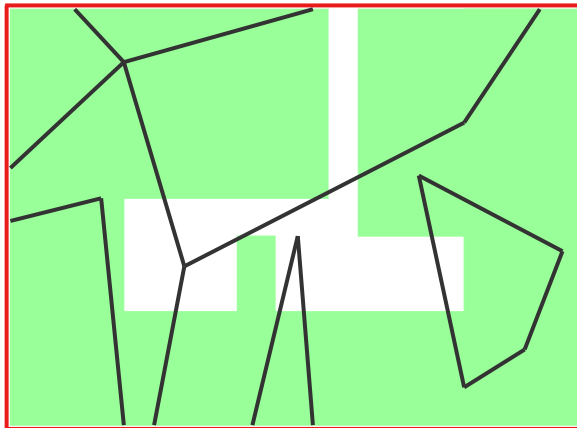
Problematizare



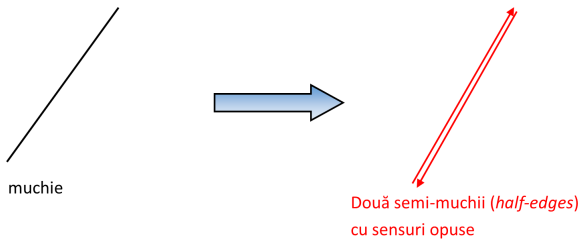
Problematizare



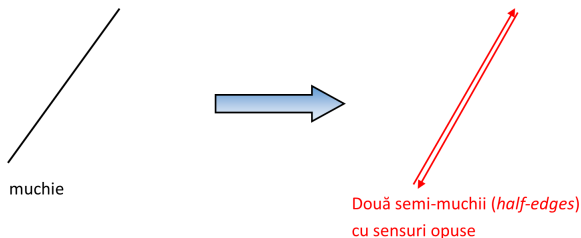
Problematizare



Ideea cheie

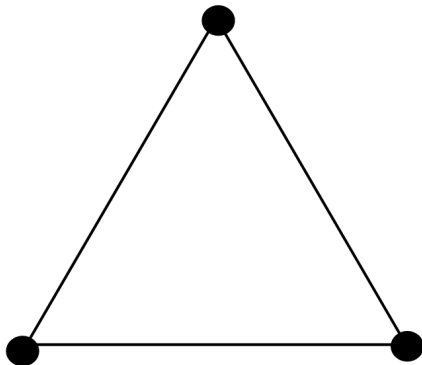


Ideea cheie

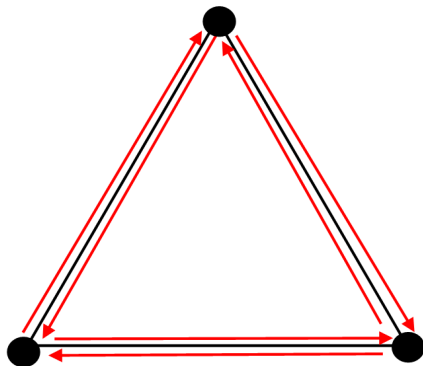


- Ideea fundamentală este de a introduce conceptul de semi-muchie (muchie orientată), cf. "*half-edge*".

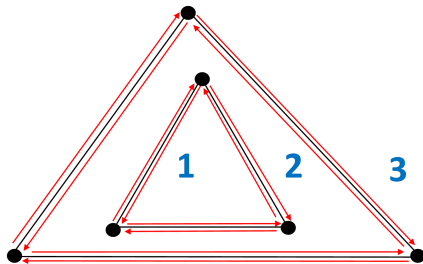
Exemplu



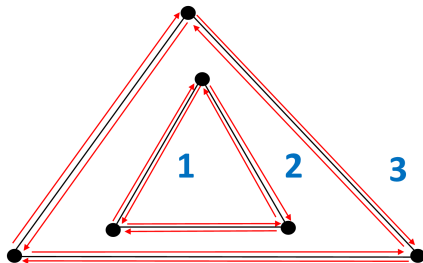
Exemplu



Exemplu

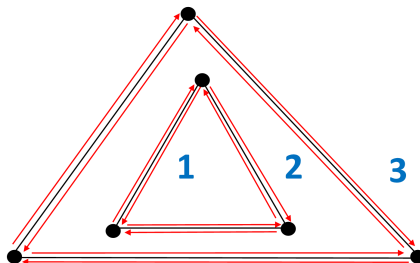


Exemplu



Subdiviziunea din figură are 6 vârfuri, 12 semi-muchii, 3 fețe.

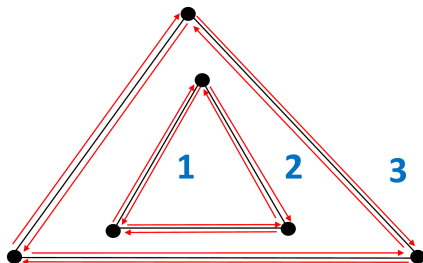
Exemplu



Subdiviziunea din figură are 6 vârfuri, 12 semi-muchii, 3 fețe.

Semi-muchiile delimitează frontiere ale fețelor, fața delimitată fiind la stânga.

Exemplu

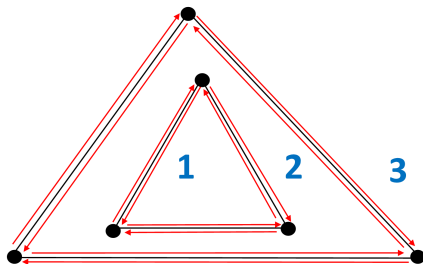


Subdiviziunea din figură are 6 vârfuri, 12 semi-muchii, 3 fețe.

Semi-muchiile delimitează frontiere ale fețelor, fața delimitată fiind la stânga.

Dat un poligon (eventual cu goluri):

Exemplu



Subdiviziunea din figură are 6 vârfuri, 12 semi-muchii, 3 fețe.

Semi-muchiile delimitează frontiere ale fețelor, fața delimitată fiind la stânga.

Dat un poligon (eventual cu goluri):

- ▶ **frontieră exterioară**, care poate fi parcursă cu ajutorul semi-muchiilor astfel încât poligonul să fie la stânga frontierei, iar virajele convexe să fie la stânga,
- ▶ **frontieră interioară** (dacă există goluri), caz în care poligonul este tot la stânga, dar virajele în vârfurile convexe sunt la dreapta.

Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.

Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).

Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
 - ▶ **Vârf** v : coordonatele lui v în $Coordinates(v)$, pointer $IncidentEdge(v)$ spre o muchie orientată care are v ca origine

Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
 - ▶ **Vârf** v : coordonatele lui v în $Coordinates(v)$, pointer $IncidentEdge(v)$ spre o muchie orientată care are v ca origine
 - ▶ **Față** f : pointer $OuterComponent(f)$ spre o muchie orientată corespunzătoare frontierei externe (pentru fața nemărginită este **nil**); listă $InnerComponents(f)$, care conține, pentru fiecare gol, un pointer către una dintre muchiile orientate de pe frontieră

Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlanțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
 - ▶ **Vârf** v : coordonatele lui v în $Coordinates(v)$, pointer $IncidentEdge(v)$ spre o muchie orientată care are v ca origine
 - ▶ **Față** f : pointer $OuterComponent(f)$ spre o muchie orientată corespunzătoare frontierei externe (pentru fața nemărginită este **nil**); listă $InnerComponents(f)$, care conține, pentru fiecare gol, un pointer către una dintre muchiile orientate de pe frontieră
 - ▶ **Muchie orientată** $\vec{e}$: pointer $Origin(\vec{e})$, pointer $Twin(\vec{e})$ pointer $IncidentFace(\vec{e})$, pointer $Next(\vec{e})$, pointer $Prev(\vec{e})$.

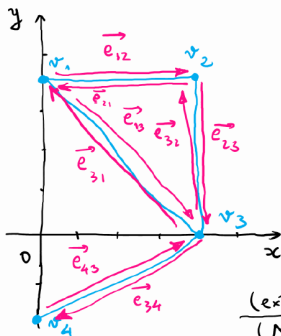
Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
 - ▶ **Vârf** v : coordonatele lui v în $Coordinates(v)$, pointer $IncidentEdge(v)$ spre o muchie orientată care are v ca origine
 - ▶ **Față** f : pointer $OuterComponent(f)$ spre o muchie orientată corespunzătoare frontierei externe (pentru fața nemărginită este **nil**); listă $InnerComponents(f)$, care conține, pentru fiecare gol, un pointer către una dintre muchiile orientate de pe frontieră
 - ▶ **Muchie orientată** $\vec{e}$: pointer $Origin(\vec{e})$, pointer $Twin(\vec{e})$ pointer $IncidentFace(\vec{e})$, pointer $Next(\vec{e})$, pointer $Prev(\vec{e})$.
- ▶ Oricărei subdiviziuni planare $\mathcal{S}$ i se asociază o listă de muchii dublu înlănțuite $\mathcal{D}_{\mathcal{S}}$.

Subdiviziuni planare

- ▶ Conceptul de subdiviziune planară: vârfuri, muchii, fețe.
- ▶ **Listă de muchii dublu înlănțuite / DCEL - Doubly Connected Edge List** [Müller și Preparata, 1978] (trei înregistrări: vârfuri, fețe, muchii orientate (semi-muchii)).
 - ▶ **Vârf** v : coordonatele lui v în $Coordinates(v)$, pointer $IncidentEdge(v)$ spre o muchie orientată care are v ca origine
 - ▶ **Față** f : pointer $OuterComponent(f)$ spre o muchie orientată corespunzătoare frontierei externe (pentru fața nemărginită este **nil**); listă $InnerComponents(f)$, care conține, pentru fiecare gol, un pointer către una dintre muchiile orientate de pe frontieră
 - ▶ **Muchie orientată** $\vec{e}$: pointer $Origin(\vec{e})$, pointer $Twin(\vec{e})$ pointer $IncidentFace(\vec{e})$, pointer $Next(\vec{e})$, pointer $Prev(\vec{e})$.
- ▶ Oricărei subdiviziuni planare $\mathcal{S}$ i se asociază o listă de muchii dublu înlănțuite $\mathcal{D}_{\mathcal{S}}$.
- ▶ **Obs.** Explicați cum, folosind pointerii de mai sus: (i) poate fi parcursă frontiera exterioară / interioară a unei fețe (a unui poligon); (ii) pot fi găsite toate semi-muchiile incidente cu un vârf.

Exemplu

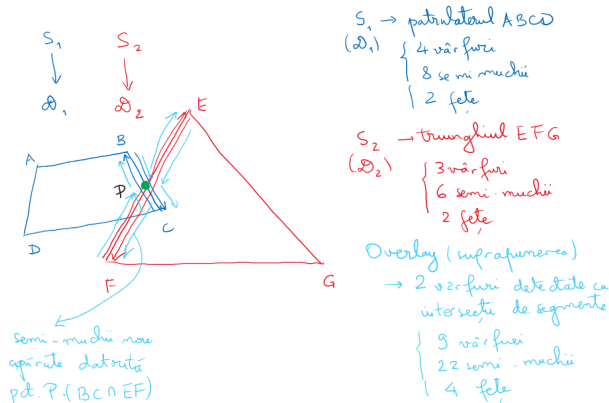


Vârf	Coordonate	Incident Edge
v_1	(0, 4)	$\overrightarrow{e_{12}}$
v_2	(4, 4)	$\overrightarrow{e_{23}}$
v_3	(4, 0)	$\overrightarrow{e_{34}}$
v_4	(0, -2)	$\overrightarrow{e_{43}}$

Față	Outer Component	Inner Component(s)
(exterior) f_1	nil	$\overrightarrow{e_{12}}$
(Δ) f_2	$\overrightarrow{e_{21}}$	nil

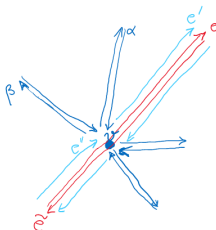
Semi-muchie	Origine	Termin	Incident Face	Next	Prev
$\overrightarrow{e_{12}}$	v_1	$\overrightarrow{e_{21}}$	f_1	$\overrightarrow{e_{23}}$	$\overrightarrow{e_{31}}$
$\overrightarrow{e_{23}}$	v_2	$\overrightarrow{e_{32}}$	f_1	$\overrightarrow{e_{34}}$	$\overrightarrow{e_{12}}$
$\overrightarrow{e_{34}}$	v_3	$\overrightarrow{e_{43}}$	f_2	$\overrightarrow{e_{43}}$	$\overrightarrow{e_{23}}$

Suprapunerea (*Overlay*) pentru subdiviziuni planare



În figură sunt considerate două subdiviziuni S_1, S_2 și structurile de tip DCEL asociate, fiind reprezentate doar semi-muchiile corespunzătoare muchiilor BC și EF . Datorită punctului de intersecție P , aceste semi-muchii vor fi înlocuite cu alte perechi de semi-muchii. În mod analog se procedează și pentru intersecția dintre CD și EF .

Actualizarea listei de semi-muchii



În exemplul din figură, prin vârful v al subdiviziunii S_1 (cu albastru) trece o muchie a subdiviziunii S_2 (cu roșu).

Actualizarea presupune definirea unor semi-muchii noi (sau păstrarea celor vechi, după caz), precum și definirea / actualizarea pointerilor asociați. La actualizare se ține cont de principiul *fața delimitată de o muchie este la stânga acesteia*.

De exemplu, semi-muchia e este înlocuită cu două semi-muchii e' și e'' . Pentru e' pointerii asociați sunt: $Origin(e')=v$, $Twin(e')$ este legat de actualizarea lui $\tilde{e}$, $Next(e')=Next(e)$, $Prev(e')=\alpha$ (semi-muchia "cea mai apropiată de e' în sens trigonometric"). Pentru e'' pointerii asociați sunt: $Origin(e'')=Origin(e)$, $Twin(e'')$ este legat de actualizarea lui $\tilde{e}$, $Next(e'')=\beta$ (semi-muchia "cea mai apropiată de e'' în sens orar"), $Prev(e'')=Prev(e)$.

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

- **Input.** Două subdiviziuni planare $\mathcal{S}_1, \mathcal{S}_2$ memorate în liste dublu înlanțuite $\mathcal{D}_{\mathcal{S}_1}, \mathcal{D}_{\mathcal{S}_2}$

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

- ▶ **Input.** Două subdiviziuni planare $\mathcal{S}_1, \mathcal{S}_2$ memorate în liste dublu înălănțuite $\mathcal{D}_{\mathcal{S}_1}, \mathcal{D}_{\mathcal{S}_2}$
- ▶ **Output.** Overlay-ul $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$ dintre $\mathcal{S}_1, \mathcal{S}_2$, memorat într-o listă dublu înălănțuită $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

- ▶ **Input.** Două subdiviziuni planare $\mathcal{S}_1, \mathcal{S}_2$ memorate în liste dublu înălțuite $\mathcal{D}_{\mathcal{S}_1}, \mathcal{D}_{\mathcal{S}_2}$
 - ▶ **Output.** Overlay-ul $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$ dintre $\mathcal{S}_1, \mathcal{S}_2$, memorat într-o listă dublu înălțuită $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$
1. Copiază listele $\mathcal{D}_1, \mathcal{D}_2$ într-o nouă listă dublu înălțuită $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

- ▶ **Input.** Două subdiviziuni planare $\mathcal{S}_1, \mathcal{S}_2$ memorate în liste dublu înălănțuite $\mathcal{D}_{\mathcal{S}_1}, \mathcal{D}_{\mathcal{S}_2}$
 - ▶ **Output.** Overlay-ul $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$ dintre $\mathcal{S}_1, \mathcal{S}_2$, memorat într-o listă dublu înălănțuită $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$
1. Copiază listele $\mathcal{D}_1, \mathcal{D}_2$ într-o nouă listă dublu înălănțuită $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$
 2. Calculează toate intersecțiile de muchii dintre $\mathcal{S}_1$ și $\mathcal{S}_2$ cu algoritmul INTERSECTII. La fiecare eveniment, pe lângă actualizarea lui $\mathcal{Q}$ și $\mathcal{T}$, efectuează:

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

- ▶ **Input.** Două subdiviziuni planare $\mathcal{S}_1, \mathcal{S}_2$ memorate în liste dublu înălănțuite $\mathcal{D}_{\mathcal{S}_1}, \mathcal{D}_{\mathcal{S}_2}$
 - ▶ **Output.** Overlay-ul $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$ dintre $\mathcal{S}_1, \mathcal{S}_2$, memorat într-o listă dublu înălănțuită $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$
1. Copiază listele $\mathcal{D}_1, \mathcal{D}_2$ într-o nouă listă dublu înălănțuită $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$
 2. Calculează toate intersecțiile de muchii dintre $\mathcal{S}_1$ și $\mathcal{S}_2$ cu algoritmul INTERSECTII. La fiecare eveniment, pe lângă actualizarea lui $\mathcal{Q}$ și $\mathcal{T}$, efectuează:
 - ▶ Actualizează $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$, în cazul în care evenimentul implică atât muchii ale lui $\mathcal{S}_1$, cât și ale lui $\mathcal{S}_2$

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

- ▶ **Input.** Două subdiviziuni planare $\mathcal{S}_1, \mathcal{S}_2$ memorate în liste dublu înălțuite $\mathcal{D}_{\mathcal{S}_1}, \mathcal{D}_{\mathcal{S}_2}$
 - ▶ **Output.** Overlay-ul $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$ dintre $\mathcal{S}_1, \mathcal{S}_2$, memorat într-o listă dublu înălțuită $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$
1. Copiază listele $\mathcal{D}_1, \mathcal{D}_2$ într-o nouă listă dublu înălțuită $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$
 2. Calculează toate intersecțiile de muchii dintre $\mathcal{S}_1$ și $\mathcal{S}_2$ cu algoritmul INTERSECTII. La fiecare eveniment, pe lângă actualizarea lui $\mathcal{Q}$ și $\mathcal{T}$, efectuează:
 - ▶ Actualizează $\mathcal{D}_{\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)}$, în cazul în care evenimentul implică atât muchii ale lui $\mathcal{S}_1$, cât și ale lui $\mathcal{S}_2$
 - ▶ Memorează noile muchii orientate adecvat

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

3. Determină ciclii de frontieră din $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$, stabilește natura (exteriori/interiori)

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

3. Determină ciclul de frontieră din $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$, stabilește natura (exteriori/interiori)
4. Construiește graful $\mathcal{G}$ ale cărui noduri corespund ciclilor de frontieră și ale cărui arce unesc fiecare ciclu corespunzând unui gol cu ciclul de la stânga vârfului cel mai din stânga și determină componentele conexe ale lui $\mathcal{G}$

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

3. Determină ciclul de frontieră din $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$, stabilește natura (exteriori/interiori)
4. Construiește graful $\mathcal{G}$ ale cărui noduri corespund ciclilor de frontieră și ale cărui arce unesc fiecare ciclu corespunzând unui gol cu ciclul de la stânga vârfului cel mai din stânga și determină componentele conexe ale lui $\mathcal{G}$
5. **for** fiecare componentă a lui $\mathcal{G}$

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

3. Determină ciclul de frontieră din $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$, stabilește natura (exteriori/interiori)
4. Construiește graful $\mathcal{G}$ ale cărui noduri corespund ciclilor de frontieră și ale cărui arce unesc fiecare ciclu corespunzând unui gol cu ciclul de la stânga vârfului cel mai din stânga și determină componentele conexe ale lui $\mathcal{G}$
5. **for** fiecare componentă a lui $\mathcal{G}$
6. **do** Fie $\mathcal{C}$ unicul ciclu de frontieră exterioară a componentei și fie f fața mărginită a ciclului. Creează o înregistrare pentru f , setează *OuterComponent*(f) (către una din muchiile lui $\mathcal{C}$), construiește lista *InnerComponents*(f) (pentru fiecare gol, pointer către una dintre muchiile orientate). Pentru fiecare muchie orientată, *IncidentFace*() către înregistrarea lui f

Algoritm SUPRAPUNERE ($\mathcal{S}_1, \mathcal{S}_2$)

3. Determină ciclul de frontieră din $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$, stabilește natura (exteriori/interiori)
4. Construiește graful $\mathcal{G}$ ale cărui noduri corespund ciclilor de frontieră și ale cărui arce unesc fiecare ciclu corespunzând unui gol cu ciclul de la stânga vârfului cel mai din stânga și determină componentele conexe ale lui $\mathcal{G}$
5. **for** fiecare componentă a lui $\mathcal{G}$
6. **do** Fie $\mathcal{C}$ unicul ciclu de frontieră exterioară a componentei și fie f fața mărginită a ciclului. Creează o înregistrare pentru f , setează *OuterComponent*(f) (către una din muchiile lui $\mathcal{C}$), construiește lista *InnerComponents*(f) (pentru fiecare gol, pointer către una dintre muchiile orientate). Pentru fiecare muchie orientată, *IncidentFace*() către înregistrarea lui f
7. Etichetează fiecare față a lui $\mathcal{O}(\mathcal{S}_1, \mathcal{S}_2)$

Rezultate principale

- **Teoremă. (Overlay-ul hărților)** Fie S_1 o subdiviziune de complexitate n_1 , S_2 o subdiviziune de complexitate n_2 , fie $n := n_1 + n_2$. Overlay-ul dintre S_1 și S_2 poate fi construit în $O(n \log n + k \log n)$, unde k este complexitatea overlay-ului.

Rezultate principale

- ▶ **Teoremă. (Overlay-ul hărților)** Fie S_1 o subdiviziune de complexitate n_1 , S_2 o subdiviziune de complexitate n_2 , fie $n := n_1 + n_2$. Overlay-ul dintre S_1 și S_2 poate fi construit în $O(n \log n + k \log n)$, unde k este complexitatea overlay-ului.
- ▶ **Corolar. (Operații boolene)** Fie $\mathcal{P}_1$ un poligon cu n_1 vârfuri și $\mathcal{P}_2$ un poligon cu n_2 vârfuri; fie $n = n_1 + n_2$. Atunci $\mathcal{P}_1 \cup \mathcal{P}_2$, $\mathcal{P}_1 \cap \mathcal{P}_2$ și $\mathcal{P}_1 \setminus \mathcal{P}_2$ pot fi determinate în timp $O(n \log n + k \log n)$, unde k este complexitatea output-ului.